DEPARTMENT OF COMPUTER APPLICATION

TKM COLLEGE ENGINEERING

KOLLAM - 691005

25MCAP105 - Advanced Data Structures

PRACTICAL RECORD BOOK

First Semester MCA

2025-26

Submitted By

NAME :

REG NO :

DEPARTMENT OF COMPUTER APPLICATION TKM COLLEGE ENGINEERING KOLLAM - 691005

CERTIFICATE

This is a bonafide record of the work done by in the First Semester for the lab course Advanced Data Structures (23MCAP105) towards the partial fulfillment of the degree of Master of Computer Applications during the academic year 2025-2026

Staff Member in-charge **Examiner**

....

INDEX

S.NO	TITLE	PAGE.NO
1.	Program to implement Array Operations using Functions	1
2.	Program to implement Merge Two Sorted Arrays	7
3.	Program to implement Stack Operations using Array	10
4.	Program to implement Queue Operations using Array	14
5.	Program to implement Browser History using Stack	18
6.	Program to implement Hospital Patient Queue using Array	22
7.	Program to implement Singly Linked List – Create, Insert, Delete	26
8.	Program to implement Phone Contacts using Singly Linked List	34
9.	Program to implement Music Player using Doubly Linked List	37
10.	Program to implement Singly Linked Stack	44
11.	Program to implement Printer Job Queue using Circular Queue	48
12.	Program to implement Binary Search Tree	52
13.	Program to implement Heap Sort	59
14.	Program to implement Graph Traversal – BFS and DFS	60
15.	Program to implement Dijkstra's Shortest Path Algorithm	63

PROGRAM 1

Implement various array operations using switch and functions:

a) Search an element in an array.

b) Sort an array in ascending and descending order.

c) Min and Max element in an array

d) Average of an array

e) Reverse an array

```
#include<stdio.h>
int minmax(int array[] , int n);
int reverse(int array[] , int n);
int search(int array[] , int n);
int sort(int array[] , int n);
int mean(int array[] , int n);

int main()
{
    int i,n,array[100],choice;
    printf("Enter limit: ");
    scanf("%d",&n);

    printf("Enter the array elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&array[i]);

    printf("Entered array is:\n");
    for(i=0;i<n;i++)
        printf("%d ",array[i]);
```

```
    printf("\n");
    printf("\nSelect array operation:\n");
    printf("1: Minimum and Maximum\n");
    printf("2: Reverse Array\n");
    printf("3: Search Element\n");
    printf("4: Sort Array (Asc & Desc)\n");
    printf("5: Mean of Array\n");
    printf("6: Exit\n");
    do {
        printf("Enter a choice:");
        scanf("%d",&choice);

        switch(choice)
        {
            case 1: minmax(array,n); break;
            case 2: reverse(array,n); break;
            case 3: search(array,n); break;
            case 4: sort(array,n); break;
            case 5: mean(array,n); break;
            case 6: printf("Exited Successfully!\n"); break;
            default: printf("Invalid choice!\n");
        }

    } while(choice != 6);
    return 0;
}

int minmax(int array[],int n)
{
    int i,min=array[0],max=array[0];
```

```
    for(i=1;i<n;i++)
    {
        if(array[i] < min) min = array[i];
        if(array[i] > max) max = array[i];
    }
    printf("Maximum element: %d\n", max);
    printf("Minimum element: %d\n", min);
    return 0;
}

int reverse(int array[], int n)
{
    printf("Reversed array:\n");
    for(int i=n-1;i>=0;i--)
        printf("%d ",array[i]);
    printf("\n");
    return 0;
}

int search(int array[], int n)
{
    int key,found=0;
    printf("Enter element to search: ");
    scanf("%d",&key);

    for(int i=0;i<n;i++)
    {
        if(array[i] == key)
        {
            printf("Element %d found at position %d (index %d)\n", key, i+1, i);
```

```
            found = 1;
        }
    }

    if(!found)
        printf("Element %d is not found\n", key);

    return 0;
}

int sort(int array[], int n)
{
    int asc[100], desc[100];
    for(int i=0; i<n; i++) {
        asc[i] = array[i];
        desc[i] = array[i];
    }

    // Sort ascending
    for(int i=0; i<n-1; i++) {
        for(int j=i+1; j<n; j++) {
            if(asc[i] > asc[j]) {
                int temp = asc[i];
                asc[i] = asc[j];
                asc[j] = temp;
            }
        }
    }

    // Sort descending
```

```
    for(int i=0; i<n-1; i++) {
        for(int j=i+1; j<n; j++) {
            if(desc[i] < desc[j]) {
                int temp = desc[i];
                desc[i] = desc[j];
                desc[j] = temp;
            }
        }
    }
    printf("Array sorted in Ascending order:\n");
    for(int i=0; i<n; i++)
        printf("%d ", asc[i]);
    printf("\n");
    printf("Array sorted in Descending order:\n");
    for(int i=0; i<n; i++)
        printf("%d ", desc[i]);
    printf("\n");

    return 0;
}

int mean(int array[],int n)
{
    int sum = 0;
    for(int i=0;i<n;i++)
        sum += array[i];

    float avg = (float)sum / n;
    printf("Mean of array: %.2f\n", avg);
    return 0;
```

```
}
```

OUTPUT

```
Enter limit: 5
Enter the array elements:
3 1 4 2 5
Entered array is:
3 1 4 2 5

Select array operation:
1: Minimum and Maximum
2: Reverse Array
3: Search Element
4: Sort Array (Asc & Desc)
5: Mean of Array
6: Exit
Enter a choice:1
Maximum element: 5
Minimum element: 1
Enter a choice:2
Reversed array:
5 2 4 1 3
Enter a choice:3
Enter element to search: 4
Element 4 found at position 3 (index 2)
Enter a choice:4
Array sorted in Ascending order:
1 2 3 4 5
Array sorted in Descending order:
5 4 3 2 1
Enter a choice:5
Mean of array: 3.00
Enter a choice:6
Exited Successfully!
```

PROGRAM-2

Merge two sorted arrays and store in a third array.

```
#include <stdio.h>
void merge(int arr[], int l, int m, int r)
 {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
```

```
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        mergeSort(arr, l, m);      // Sort first half
        mergeSort(arr, m + 1, r);  // Sort second half
        merge(arr, l, m, r);       // Merge halves
    }
}

// Function to merge two sorted arrays
void mergeTwoArrays(int a[], int n1, int b[], int n2, int c[]) {
    int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2) {
        if (a[i] <= b[j])
            c[k++] = a[i++];
        else
            c[k++] = b[j++];
    }

    while (i < n1)
        c[k++] = a[i++];

    while (j < n2)
        c[k++] = b[j++];
}
```

```
int main()
{
    int a[50], b[50], c[100];
    int n1, n2, i;

    printf("Enter size of first array: ");
    scanf("%d", &n1);
    printf("Enter %d elements:\n", n1);
    for (i = 0; i < n1; i++)
        scanf("%d", &a[i]);

    printf("Enter size of second array: ");
    scanf("%d", &n2);
    printf("Enter %d elements:\n", n2);
    for (i = 0; i < n2; i++)
        scanf("%d", &b[i]);

    // Sort both using merge sort
    mergeSort(a, 0, n1 - 1);
    mergeSort(b, 0, n2 - 1);

    // Merge the sorted arrays into a third array
    mergeTwoArrays(a, n1, b, n2, c);

    printf("\nSorted First Array: ");
    for (i = 0; i < n1; i++)
        printf("%d ", a[i]);

    printf("\nSorted Second Array: ");
    for (i = 0; i < n2; i++)
```

```
        printf("%d ", b[i]);

    printf("\nMerged Sorted Array: ");
    for (i = 0; i < n1 + n2; i++)
        printf("%d ", c[i]);

    printf("\n");
    return 0;
}
```

OUTPUT

```
Enter size of first array: 3
Enter 3 elements:
4 1 6
Enter size of second array: 3
Enter 3 elements:
5 2 3

Sorted First Array: 1 4 6
Sorted Second Array: 2 3 5
Merged Sorted Array: 1 2 3 4 5 6
```

PROGRAM-3

Implement Stack operations using array.

```
#include <stdio.h>
#define SIZE 100

int stack[SIZE];
int top = -1;
```

```
void push();
void pop();
void peek();
void display();

int main() {
    int choice;

    do {
        printf("\n*** STACK MENU ***\n");
        printf("1. PUSH\n");
        printf{ "2. POP\n");
        printf("3. PEEK\n");
        printf("4. DISPLAY\n");
        printf("5. EXIT\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
            case 4:
```

```
                display();
                break;
            case 5:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
                break;
        }

    } while(choice != 5);

    return 0;
}

void push() {
    int value;
    if (top == SIZE - 1) {
        printf("STACK OVERFLOW! Cannot push.\n");
    } else {
        printf("Enter value to push: ");
        scanf("%d", &value);
        top++;
        stack[top] = value;
        printf("%d pushed to stack.\n", value);
    }
}

void pop() {
    if (top == -1) {
```

```
        printf("STACK UNDERFLOW! No elements to pop.\n");
    } else {
        printf("%d popped from stack.\n", stack[top]);
        top--;
    }
}

void peek() {
    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Top element = %d\n", stack[top]);
    }
}

void display() {
    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Stack elements (top to bottom): ");
        for (int i = top; i >= 0; i--) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}
```

OUTPUT

```
*** STACK MENU ***
1. PUSH
2. POP
3. PEEK
4. DISPLAY
5. EXIT
Enter your choice: 1
Enter value to push: 5
5 pushed to stack.
Enter your choice: 1
Enter value to push: 10
10 pushed to stack.
Enter your choice: 1
Enter value to push: 15
15 pushed to stack.
Enter your choice: 3
Top element = 15
Enter your choice: 4
Stack elements (top to bottom): 15 10 5
Enter your choice: 2
15 popped from stack.
Enter your choice: 4
Stack elements (top to bottom): 10 5
Enter your choice: 5
Exiting...
```

PROGRAM-4

Implement a different queue operations using array.

```
#include <stdio.h>
#define N 100
int queue[N];
int front = -1, rear = -1;
void enqueue();
void dequeue();
void peek();
void display();

int main() {
    int choice;
```

```
    do {
        printf("\n*** QUEUE MENU ***\n");
        printf("1. ENQUEUE\n");
        printf("2. DEQUEUE\n");
        printf("3. PEEK\n");
        printf("4. DISPLAY\n");
        printf("5. EXIT\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }

    } while(choice != 5);

    return 0;
}

void enqueue() {
    int value;
    if (rear == N - 1) {
        printf("Queue Overflow!\n");
    } else {
```

```
        printf("Enter value to insert: ");
        scanf("%d", &value);
        if (front == -1) front = 0;
        rear++;
        queue[rear] = value;
        printf("%d inserted into queue.\n", value);
    }
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow!\n");
    } else {
        printf("%d removed from queue.\n", queue[front]);
        front++;
        if (front > rear) front = rear = -1;
    }
}

void peek() {
    if (front == -1) {
        printf("Queue is empty.\n");
    } else {
        printf("Front element = %d\n", queue[front]);
    }
}

void display() {
    if (front == -1) {
        printf("Queue is empty.\n");
```

```
    } else {
        printf("Queue elements: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}
```

OUTPUT

```
*** QUEUE MENU ***
1. ENQUEUE
2. DEQUEUE
3. PEEK
4. DISPLAY
5. EXIT
Enter your choice: 1
Enter value to insert: 5
5 inserted into queue.
Enter your choice: 1
Enter value to insert: 10
10 inserted into queue.
Enter your choice: 1
Enter value to insert: 15
15 inserted into queue.
Enter your choice: 4
Queue elements: 5 10 15
Enter your choice: 3
Front element = 5
Enter your choice: 2
5 removed from queue.
Enter your choice: 4
Queue elements: 10 15
Enter your choice: 5
Exiting...
```

PROGRAM-5

Implement a program to push, pop, top and display the history of visited websites in a stack using array.

```
#include <stdio.h>
#include <string.h>
#define N 100

char history[N][50];
int top = -1;

void push();
void pop();
void topWebsite();
void display();

int main() {
    int choice;

    printf("\n*** BROWSER HISTORY MENU ***\n");
    printf("1. Visit New Website (PUSH)\n");
    printf("2. Go Back (POP)\n");
    printf("3. Current Website (TOP)\n");
    printf("4. Show History (DISPLAY)\n");
    printf("5. EXIT\n");

    do {
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
```

```
        switch(choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: topWebsite(); break;
            case 4: display(); break;
            case 5: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }

    } while(choice != 5);

    return 0;
}

void push() {
    char site[50];
    if (top == N - 1) {
        printf("History is full! Cannot visit new website.\n");
    } else {
        printf("Enter website name: ");
        scanf("%s", site);
        top++;
        strcpy(history[top], site);
        printf("Visited: %s\n", site);
    }
}

void pop() {
    if (top == -1) {
```

```
        printf("No browsing history! Cannot go back.\n");
    } else {
        printf("Going back from: %s\n", history[top]);
        top--;
    }
}

void topWebsite() {
    if (top == -1) {
        printf("No website currently open.\n");
    } else {
        printf("Current website: %s\n", history[top]);
    }
}

void display() {
    if (top == -1) {
        printf("No browsing history.\n");
    } else {
        printf("Browsing History (Newest to Oldest):\n");
        for (int i = top; i >= 0; i--) {
            printf("%s\n", history[i]);
        }
    }
}
```

OUTPUT

```
*** BROWSER HISTORY MENU ***
1. Visit New Website (PUSH)
2. Go Back (POP)
3. Current Website (TOP)
4. Show History (DISPLAY)
5. EXIT

Enter your choice: 1
Enter website name: google
Visited: google
Enter your choice: 1
Enter website name: youtube
Visited: youtube
Enter your choice: 1
Enter website name: yahoo
Visited: yahoo
Enter your choice: 3
Current website: yahoo
Enter your choice: 4
Browsing History:
yahoo   youtube google
Enter your choice: 2
Going back from: yahoo
Enter your choice: 4
Browsing History:
youtube google
Enter your choice: 5
Exiting...
```

PROGRAM-6

A hospital has a queue system to manage patients waiting for consultation. Patients arrive and are added to the waiting list. The doctor calls patients in the order they arrived. C program to implement this system using an array-based queue with the following operations:

a. Enqueue: Add a new patient to the queue.

b. Dequeue: Remove the patient who is first in the queue (consulted by the doctor).

c. Top/Peek: Display the name of the next patient to be consulted without removing them.

d. Display: Show all patients currently in the waiting list

```
#include <stdio.h>
#include <string.h>
#define N 100
char queue[N][50];
int front = -1, rear = -1;
void enqueue();
void dequeue();
void peek();
void display();

int main() {
      int choice;
      printf("\n*** HOSPITAL QUEUE SYSTEM ***\n");
      printf("1.\tAdd Patient (ENQUEUE)\n");
      printf("2.\tConsult Patient (DEQUEUE)\n");
```

```
	printf("3.\tNext Patient (PEEK)\n");
	printf("4.\tShow Waiting List (DISPLAY)\n");
	printf("5.\tEXIT\n");

	do {
		printf("\nEnter your choice:\t");
		scanf("%d", &choice);

		switch(choice) {
			case 1: enqueue(); break;
			case 2: dequeue(); break;
			case 3: peek(); break;
			case 4: display(); break;
			case 5: printf("Exiting...\n"); break;
			default: printf("Invalid choice!\n");
		}
	} while(choice != 5);

	return 0;
}

void enqueue() {
	char name[50];
	if (rear == N - 1) {
		printf("Queue Full! Cannot add more patients.\n");
	} else {
		printf("Enter patient name:\t");
		scanf("%s", name);
		if (front == -1) front = 0;
		rear++;
```

```
            strcpy(queue[rear], name);
            printf("Patient added:\t%s\n", name);
      }
}

void dequeue() {
      if (front == -1 || front > rear) {
            printf("No patients in the waiting list.\n");
      } else {
            printf("Patient consulted:\t%s\n", queue[front]);
            front++;
            if (front > rear) front = rear = -1;
      }
}

void peek() {
      if (front == -1) {
            printf("No patients waiting.\n");
      } else {
            printf("Next patient:\t%s\n", queue[front]);
      }
}

void display() {
      if (front == -1) {
            printf("No patients in the waiting list.\n");
      } else {
            printf("Waiting List:\n");
            for (int i = front; i <= rear; i++) {
                  printf("\t%s\n", queue[i]);
```

```
            }
        }
}
```

OUTPUT

```
*** HOSPITAL QUEUE SYSTEM ***
1.  Add Patient (ENQUEUE)
2.  Consult Patient (DEQUEUE)
3.  Next Patient (PEEK)
4.  Show Waiting List (DISPLAY)
5.  EXIT

Enter your choice:  1
Enter patient name: Pranav
Patient added:  Pranav

Enter your choice:  1
Enter patient name: Arun
Patient added:  Arun

Enter your choice:  1
Enter patient name: Sachin
Patient added:  Sachin

Enter your choice:  3
Next patient:   Pranav

Enter your choice:  4
Waiting List:
    Pranav
    Arun
    Sachin

Enter your choice:  2
Patient consulted:  Pranav

Enter your choice:  4
Waiting List:
    Arun
    Sachin

Enter your choice:  5
Exiting...
```

PROGRAM-7

Implement creation, insertion (3 cases) and deletion (3 cases) in a single linked list.

```
#include<stdio.h>
#include<stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL, *temp, *newnode;

void insert1();
void insert2();
void insert3();
void delete1();
void delete2();
void delete3();
void display();

int main() {
    int x;

    printf("Creating a linked list:\n");
    do {
        newnode = (struct node*)malloc(sizeof(struct node));
        printf("Enter data:\n");
```

```
        scanf("%d", &newnode->data);
        newnode->next = NULL;

        if (head == NULL) {
            head = temp = newnode;
        } else {
            temp->next = newnode;
            temp = temp->next;
        }

        printf("Do you want to add new node (0/1)? ");
        scanf("%d", &x);

    } while (x != 0);

    printf("Created linked list is:\n");
    display();

    do {
        printf("\nEnter a choice:\n");
        printf("1) Insert at beginning\n2) Insert at middle\n3) Insert at
end\n");
        printf("4) Delete at beginning\n5) Delete at middle\n6) Delete at
end\n");
        printf("7) Display\n8) Exit\n");
        scanf("%d", &x);

        switch (x) {
            case 1: insert1(); display(); break;
            case 2: insert2(); display(); break;
            case 3: insert3(); display(); break;
```

```
            case 4: delete1(); display(); break;
            case 5: delete2(); display(); break;
            case 6: delete3(); display(); break;
            case 7: display(); break;
            case 8: printf("Exiting...\n"); break;
            default: printf("Enter a valid choice\n");
        }

    } while (x != 8);

    return 0;
}

void insert1() {
    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter data: ");
    scanf("%d", &newnode->data);
    newnode->next = head;
    head = newnode;
}

void insert2() {
    int x;
    struct node *p;

    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter data: ");
    scanf("%d", &newnode->data);

    printf("Enter the node value after which insertion should happen: ");
```

```
    scanf("%d", &x);

    temp = head;
    while (temp != NULL && temp->data != x) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Node not found – insertion failed.\n");
        free(newnode);
        return;
    }

    newnode->next = temp->next;
    temp->next = newnode;
}

void insert3() {
    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter data: ");
    scanf("%d", &newnode->data);
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
        return;
    }

    temp = head;
    while (temp->next != NULL)
```

```
        temp = temp->next;

    temp->next = newnode;
}

void delete1() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    head = head->next;
    printf("Node %d deleted.\n", temp->data);
    free(temp);
}

void delete2() {
    int x;
    struct node *p = NULL;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Enter node value to delete: ");
    scanf("%d", &x);

    temp = head;
```

```
    while (temp != NULL && temp->data != x) {
        p = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Node not found.\n");
        return;
    }

    if (p == NULL) {
        delete1();
        return;
    }

    p->next = temp->next;
    printf("Node %d deleted.\n", temp->data);
    free(temp);
}

void delete3() {
    struct node *p = NULL;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
```

```
    if (head->next == NULL) {
        printf("Node %d deleted.\n", head->data);
        free(head);
        head = NULL;
        return;
    }

    while (temp->next != NULL) {
        p = temp;
        temp = temp->next;
    }

    printf("Node %d deleted.\n", temp->data);
    p->next = NULL;
    free(temp);
}

void display() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    while (temp != NULL) {
        printf("%d ---> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

OUTPUT

```
Creating a linked list:
Enter data:
3
Do you want to add new node (0/1)? 1
Enter data:
5
Do you want to add new node (0/1)? 0
Created linked list is:
3 ---> 5 ---> NULL
1) Insert at beginning
2) Insert at middle
3) Insert at end
4) Delete at beginning
5) Delete at middle
6) Delete at end
7) Display
8) Exit

Enter a choice:
1
Enter data: 1
1 ---> 3 ---> 5 ---> NULL

Enter a choice:
2
Enter data: 4
Enter the node value after which insertion should happen: 3
1 ---> 3 ---> 4 ---> 5 ---> NULL

Enter a choice:
3
Enter data: 6
1 ---> 3 ---> 4 ---> 5 ---> 6 ---> NULL

Enter a choice:
4
Node 1 deleted.
3 ---> 4 ---> 5 ---> 6 ---> NULL

Enter a choice:
5
Enter node value to delete: 5
Node 5 deleted.
3 ---> 4 ---> 6 ---> NULL

Enter a choice:
6
Node 6 deleted.
3 ---> 4 ---> NULL

Enter a choice:
8
Exiting...
```

PROGRAM-8

Implement a program to store and display contacts on a phone using Singly LL.

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int number;
    char name[50];
    struct node *next;
};

struct node *head = NULL, *temp = NULL, *newnode = NULL;

void addContact();
void display();

int main()
{
    int x;

    do
    {
        printf("\nEnter a choice:\n");
        printf("1) Add Contact\n2) Display\n3) Exit\n");
        scanf("%d", &x);

        switch(x)
```

```
        {
            case 1: addContact(); break;
            case 2: display(); break;
            case 3: printf("Exiting...\n"); break;
            default: printf("Enter a valid choice\n");
        }

    } while(x != 3);

    return 0;
}

void addContact()
{
    newnode = (struct node*)malloc(sizeof(struct node));

    printf("Enter phone number: ");
    scanf("%d", &newnode->number);

    printf("Enter name: ");
    scanf("%s", newnode->name);

    newnode->next = NULL;

    if (head == NULL)
    {
        head = newnode;
    }
    else
    {
```

```
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newnode;
    }

    printf("Contact added successfully!\n");
}

void display()
{
    if (head == NULL)
    {
        printf("The contact list is empty\n");
        return;
    }

    temp = head;
    printf("\nName\tPhone Number\n");

    while (temp != NULL)
    {
        printf("%s\t%d\n", temp->name, temp->number);
        temp = temp->next;
    }
}
```

OUTPUT

```
1) Add Contact
2) Display
3) Exit

Enter a choice:
1
Enter phone number: 9876543210
Enter name: Pranav
Contact added successfully!

Enter a choice:
1
Enter phone number: 9988776655
Enter name: Rahul
Contact added successfully!

Enter a choice:
2

Name     Phone Number
Pranav   1286608618
Rahul    1398842063

Enter a choice:
3
Exiting...
```

PROGRAM-9

Implement a program to manage music player application using DLL.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node {
    struct node *prev;
    char music[50];
    struct node *next;
};

struct node *head = NULL, *tail = NULL, *current = NULL, *temp = NULL;
```

```
void addmusic();
void deletemusic();
void display();
void previous();
void next();

int main() {
    int ch, c;

    do {
        struct node *new = (struct node*)malloc(sizeof(struct node));
        printf("Enter music: ");
        scanf("%s", new->music);
        new->prev = NULL;
        new->next = NULL;

        if (head == NULL) {
            head = tail = new;
        } else {
            tail->next = new;
            new->prev = tail;
            tail = new;
        }

        printf("Add another music? (1/0): ");
        scanf("%d", &c);
    } while (c == 1);

    current = head;
```

```
    do {
        printf("\n1) Add music\n2) Delete music\n3) Previous music\n4) Next
music\n5) Display playlist\n6) Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1: addmusic(); break;
            case 2: deletemusic(); break;
            case 3: previous(); break;
            case 4: next(); break;
            case 5: display(); break;
            case 6: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (ch != 6);

    return 0;
}

void addmusic() {
    struct node *new = (struct node*)malloc(sizeof(struct node));
    printf("Enter new music: ");
    scanf("%s", new->music);
    new->prev = NULL;
    new->next = NULL;

    if (head == NULL) {
        head = tail = current = new;
    } else {
```

```
        tail->next = new;
        new->prev = tail;
        tail = new;
    }

    printf("Music '%s' added.\n", new->music);
}

void deletemusic() {
    if (head == NULL) {
        printf("Playlist is empty!\n");
        return;
    }

    char m[50];
    printf("Enter music to delete: ");
    scanf("%s", m);

    struct node *p = head;

    while (p != NULL && strcmp(p->music, m) != 0)
        p = p->next;

    if (p == NULL) {
        printf("Music not found!\n");
        return;
    }

    if (current == p) {
        if (p->next != NULL)
```

```
            current = p->next;
        else
            current = p->prev;
    }

    if (p == head)
        head = p->next;
    else
        p->prev->next = p->next;

    if (p == tail)
        tail = p->prev;
    else
        p->next->prev = p->prev;

    printf("Deleted: %s\n", p->music);
    free(p);

    if (head == NULL)
        current = NULL;
}

void display() {
    if (head == NULL) {
        printf("Playlist is empty!\n");
        return;
    }

    struct node *p = head;
    printf("\n--- PLAYLIST ---\n");
```

```
    while (p != NULL) {
        if (p == current)
            printf("%s (Now Playing)\n", p->music);
        else
            printf("%s\n", p->music);

        p = p->next;
    }
}

void next() {
    if (current == NULL) {
        printf("Playlist is empty!\n");
        return;
    }

    if (current->next == NULL) {
        printf("Already at last music.\n");
        return;
    }

    current = current->next;
    printf("Now Playing: %s\n", current->music);
}

void previous() {
    if (current == NULL) {
        printf("Playlist is empty!\n");
        return;
```

```
    }

    if (current->prev == NULL) {
        printf("Already at first music.\n");
        return;
    }

    current = current->prev;
    printf("Now Playing: %s\n", current->music);
}
```

OUTPUT

```
Enter music: ShapeOfYou
Add another music? (1/0): 1
Enter music: Believer
Add another music? (1/0): 0

1) Add music
2) Delete music
3) Previous music
4) Next music
5) Display playlist
6) Exit

Enter your choice: 5

--- PLAYLIST ---
ShapeOfYou (Now Playing)
Believer

Enter your choice: 1
Enter new music: Alone

Music 'Alone' added.

Enter your choice: 4
Now Playing: Believer

Enter your choice: 3
Now Playing: ShapeOfYou

Enter your choice: 2
Enter music to delete: Alone
Deleted: Alone

Enter your choice: 5

--- PLAYLIST ---
ShapeOfYou (Now Playing)
Believer

Enter your choice: 6
Exiting...
```

PROGRAM-10

Singly Linked Stack - Push, Pop, Linear Search and count the nodes in the list.

```
#include<stdio.h>
#include<stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL, *temp, *newnode;

void push();
void pop();
void search();
void count();
void display();

int main() {
    int c;

    do {
        printf("\nEnter a choice:");
        printf("\n1)Push\n2)Pop\n3)Search\n4)Count\n5)Display\n6)Exit\n");
        scanf("%d", &c);

        switch(c) {
```

```
            case 1: push(); break;
            case 2: pop(); break;
            case 3: search(); break;
            case 4: count(); break;
            case 5: display(); break;
            case 6: printf("Exiting..\n"); break;
            default: printf("Please enter a valid choice\n");
        }

    } while(c != 6);

    return 0;
}

void push() {
    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter data: ");
    scanf("%d", &newnode->data);

    newnode->next = head;
    head = newnode;

    printf("%d pushed\n", newnode->data);
}

void pop() {
    if (head == NULL) {
        printf("Stack is empty\n");
    } else {
        temp = head;
```

```
        printf("Popped: %d\n", temp->data);
        head = head->next;
        free(temp);
    }
}

void search() {
    if (head == NULL) {
        printf("Stack is empty\n");
        return;
    }

    int x, pos = 1, found = 0;
    printf("Enter element to search: ");
    scanf("%d", &x);

    temp = head;
    while (temp != NULL) {
        if (temp->data == x) {
            printf("Element %d found at position %d from top\n", x, pos);
            found = 1;
            break;
        }
        pos++;
        temp = temp->next;
    }

    if (!found)
        printf("Element not found\n");
}
```

```
void display() {
    if (head == NULL) {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements (top to bottom): ");
    temp = head;
    while (temp != NULL) {
        printf("%d --> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void count() {
    int cnt = 0;
    temp = head;

    while (temp != NULL) {
        cnt++;
        temp = temp->next;
    }

    printf("Number of nodes in the stack: %d\n", cnt);
}
```

OUTPUT

```
1)Push
2)Pop
3)Search
4)Count
5)Display
6)Exit

Enter a choice:1
Enter data: 5
5 pushed

Enter a choice:1
Enter data: 10
10 pushed

Enter a choice:1
Enter data: 15
15 pushed

Enter a choice:5
Stack elements (top to bottom): 15 --> 10 --> 5 --> NULL

Enter a choice:3
Enter element to search: 10
Element 10 found at position 2 from top

Enter a choice:4
Number of nodes in the stack: 3

Enter a choice:2
Popped: 15

Enter a choice:5
Stack elements (top to bottom): 10 --> 5 --> NULL

Enter a choice:6
Exiting..
```

PROGRAM-11

Consider a scenario where you are developing a simulation for a printer job queue in an operating system. The printer has the capability to print multiple jobs in a sequential manner. To efficiently manage the print jobs, you decide to implement a circular queue. Provide a step-by-step implementation of the circular queue data structure that include necessary methods/functions for enqueue and dequeue.

```
#include <stdio.h>

#define MAX 5   // Queue size


int queue[MAX];
```

```
int front = -1, rear = -1;

int isFull() {
    return (front == (rear + 1) % MAX);
}

int isEmpty() {
    return (front == -1);
}

void enqueue(int job) {
    if (isFull()) {
        printf("Queue Overflow! Cannot add new print job.\n");
        return;
    }

    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % MAX;
    }

    queue[rear] = job;
    printf("Job %d added to print queue.\n", job);
}

int dequeue() {
    if (isEmpty()) {
        printf("Queue Underflow! No jobs to process.\n");
        return -1;
```

```
    }

    int job = queue[front];

    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX;
    }

    printf("Job %d processed.\n", job);
    return job;
}

void display() {
    if (isEmpty()) {
        printf("Print queue is empty.\n");
        return;
    }

    printf("Current Print Jobs: ");
    int i = front;

    while (1) {
        printf("%d ", queue[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }

    printf("\n");
```

```
}

int main() {
    int choice, job;

    while (1) {
        printf("\n--- Printer Job Queue (Circular Queue) ---\n");
        printf("1. Add Print Job (Enqueue)\n");
        printf("2. Process Job (Dequeue)\n");
        printf("3. Display Queue\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter job number: ");
                scanf("%d", &job);
                enqueue(job);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
```

```
                return 0;

            default:
                printf("Invalid choice!\n");
        }
    }
}
```

OUTPUT

```
--- Printer Job Queue (Circular Queue) ---
1. Add Print Job (Enqueue)
2. Process Job (Dequeue)
3. Display Queue
4. Exit
Enter choice: 1
Enter job number: 101
Job 101 added to print queue.
Enter choice: 1
Enter job number: 102
Job 102 added to print queue.
Enter choice: 1
Enter job number: 103
Job 103 added to print queue.
Enter choice: 3
Current Print Jobs: 101 102 103
Enter choice: 2
Job 101 processed.
Enter choice: 3
Current Print Jobs: 102 103
Enter choice: 4
```

PROGRAM-12

Implement a program to create, insert, delete and traverse a binary search tree.

```
class BinarySearchTreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
def add_child(self, data):
    if data == self.data:
        return  # node already exists, ignore duplicates

    if data < self.data:
        if self.left:
            self.left.add_child(data)
        else:
            self.left = BinarySearchTreeNode(data)
    else:
        if self.right:
            self.right.add_child(data)
        else:
            self.right = BinarySearchTreeNode(data)

def search(self, val):
    if self.data == val:
        return True

    if val < self.data:
        if self.left:
            return self.left.search(val)
        else:
            return False

    if val > self.data:
        if self.right:
            return self.right.search(val)
        else:
```

```
            return False

    def find_min(self):
        current = self
        while current.left:
            current = current.left
        return current.data

    def delete(self, val):
        # standard BST deletion, returns new root of this subtree
        if val < self.data:
            if self.left:
                self.left = self.left.delete(val)
        elif val > self.data:
            if self.right:
                self.right = self.right.delete(val)
        else:
            # node to delete found
            # case 1: no child
            if self.left is None and self.right is None:
                return None
            # case 2: one child
            if self.left is None:
                return self.right
            if self.right is None:
                return self.left
            # case 3: two children
            min_val = self.right.find_min()
            self.data = min_val
            self.right = self.right.delete(min_val)
```

```
        return self

    def in_order_traversal(self):
        elements = []
        if self.left:
            elements += self.left.in_order_traversal()
        elements.append(self.data)
        if self.right:
            elements += self.right.in_order_traversal()
        return elements

    def pre_order_traversal(self):
        elements = []
        elements.append(self.data)
        if self.left:
            elements += self.left.pre_order_traversal()
        if self.right:
            elements += self.right.pre_order_traversal()
        return elements

    def post_order_traversal(self):
        elements = []
        if self.left:
            elements += self.left.post_order_traversal()
        if self.right:
            elements += self.right.post_order_traversal()
        elements.append(self.data)
        return elements
```

```
def build_tree_from_input():
    n = int(input("Enter number of elements in initial tree: "))
    print("Enter", n, "elements (space separated):")
    arr = list(map(int, input().split()))
    # if user enters more than n, use only first n
    arr = arr[:n]
    root = BinarySearchTreeNode(arr[0])
    for i in range(1, len(arr)):
        root.add_child(arr[i])
    return root

if __name__ == '__main__':
    root = build_tree_from_input()

    while True:
        print("\n--- BINARY SEARCH TREE MENU ---")
        print("1. Insert")
        print("2. Delete")
        print("3. Inorder Traversal")
        print("4. Preorder Traversal")
        print("5. Postorder Traversal")
        print("6. Search")
        print("7. Exit")
        choice = int(input("Enter your choice: "))

        if choice == 1:
            val = int(input("Enter value to insert: "))
            if root is None:
                root = BinarySearchTreeNode(val)
            else:
```

```
            root.add_child(val)
        print("Inserted", val)

    elif choice == 2:
        if root is None:
            print("Tree is empty. Nothing to delete.")
        else:
            val = int(input("Enter value to delete: "))
            root = root.delete(val)
            print("Deleted", val, "(if it existed)")

    elif choice == 3:
        if root is None:
            print("Tree is empty.")
        else:
            print("Inorder Traversal:", root.in_order_traversal())

    elif choice == 4:
        if root is None:
            print("Tree is empty.")
        else:
            print("Preorder Traversal:", root.pre_order_traversal())

    elif choice == 5:
        if root is None:
            print("Tree is empty.")
        else:
            print("Postorder Traversal:", root.post_order_traversal())

    elif choice == 6:
```

```
        if root is None:
            print("Tree is empty.")
        else:
            val = int(input("Enter value to search: "))
            if root.search(val):
                print(val, "found in tree.")
            else:
                print(val, "not found in tree.")
    elif choice == 7:
        print("Exiting...")
        break
    else:
        print("Invalid choice. Try again.")
```

OUTPUT

```
Enter number of elements in initial tree: 6
Enter 6 elements:
10 5 21 4 11 9

--- BINARY SEARCH TREE MENU ---
1. Insert
2. Delete
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Search
7. Exit
Enter your choice: 1
Enter value to insert: 6
Inserted 6
Enter your choice: 3
Inorder Traversal: [4, 5, 6, 9, 10, 11, 21]
Enter your choice: 4
Preorder Traversal: [10, 5, 4, 9, 6, 21, 11]
Enter your choice: 5
Postorder Traversal: [4, 6, 9, 5, 11, 21, 10]
Enter your choice: 6
Enter value to search: 4
4 found in tree.
Enter your choice: 2
Enter value to delete: 9
Deleted 9
Enter your choice: 7
Exiting...
```

PROGRAM-13

Implement a program for heap sort.

```
import heapq

arr =list(map(int,input("Enter a list of elements:")))

# convert list into min heap
heapq.heapify(arr)

sorted_list = []

# pop elements one by one
while len(arr) > 0:
    x = heapq.heappop(arr)
    sorted_list.append(x)

print("Sorted list:", sorted_list)
```

OUTPUT

```
Enter a list of elements:11 5 8 3 6 1
Sorted list: [1, 3, 5, 6, 8, 11]
```

PROGRAM-14

Implement BFS and DFS graph traversal.

```
from collections import deque

# Predefined adjacency list for the graph
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

def bfs(start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    print("BFS Traversal:", end=" ")

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
```

```
                visited.add(neighbour)
                queue.append(neighbour)
    print()

def dfs(start, visited=None):
    if visited is None:
        visited = set()
        print("DFS Traversal:", end=" ")

    print(start, end=" ")
    visited.add(start)

    for neighbour in graph[start]:
        if neighbour not in visited:
            dfs(neighbour, visited)

# Main program
start = 'A'  # Choose starting node
print("\n--- GRAPH TRAVERSAL MENU ---")
print("1. BFS Traversal")
print("2. DFS Traversal")
print("3. Display Graph")
print("4. Exit")
while True:

    choice = int(input("Enter your choice: "))

    if choice == 1:
        bfs(start)
```

```
elif choice == 2:
    dfs(start)
    print()

elif choice == 3:
    print("\nAdjacency List:")
    for v in graph:
        print(v, ":", graph[v])

elif choice == 4:
    print("Exiting...")
    break
else:
    print("Invalid choice, try again.")
```

OUTPUT

```
--- GRAPH TRAVERSAL MENU ---
1. BFS Traversal
2. DFS Traversal
3. Display Graph
4. Exit
Enter your choice: 3

Adjacency List:
A : ['B', 'C']
B : ['A', 'D', 'E']
C : ['A', 'F']
D : ['B']
E : ['B', 'F']
F : ['C', 'E']
Enter your choice: 1
BFS Traversal: A B C D E F
Enter your choice: 2
DFS Traversal: A B D E F C
Enter your choice: 4
Exiting...
```

PROGRAM-15

Implement Dijkstra's shortest path program.

```
import heapq

def dijkstra(graph, start):
    # Step 1: Initialize all distances as infinity
    distance = {node: float('inf') for node in graph}
    distance[start] = 0

    # Priority queue with (distance, node)
    pq = [(0, start)]

    while pq:
        dist, node = heapq.heappop(pq)

        # Skip if we already found a better path
        if dist > distance[node]:
            continue

        # Check all neighbors
        for neighbor, weight in graph[node]:
            new_dist = dist + weight

            # If a shorter path is found, update it
            if new_dist < distance[neighbor]:
                distance[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))

    return distance
```

```
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('A', 2), ('C', 1), ('D', 7)],
    'C': [('A', 4), ('B', 1), ('D', 3)],
    'D': [('B', 7), ('C', 3)]
}



start_node = 'A'
result = dijkstra(graph, start_node)

print("Shortest distances from node", start_node)
for node, dist in result.items():
    print(f"{node} → {dist}")
```

OUTPUT

```
Shortest distances from node A
A → 0
B → 2
C → 3
D → 6
```